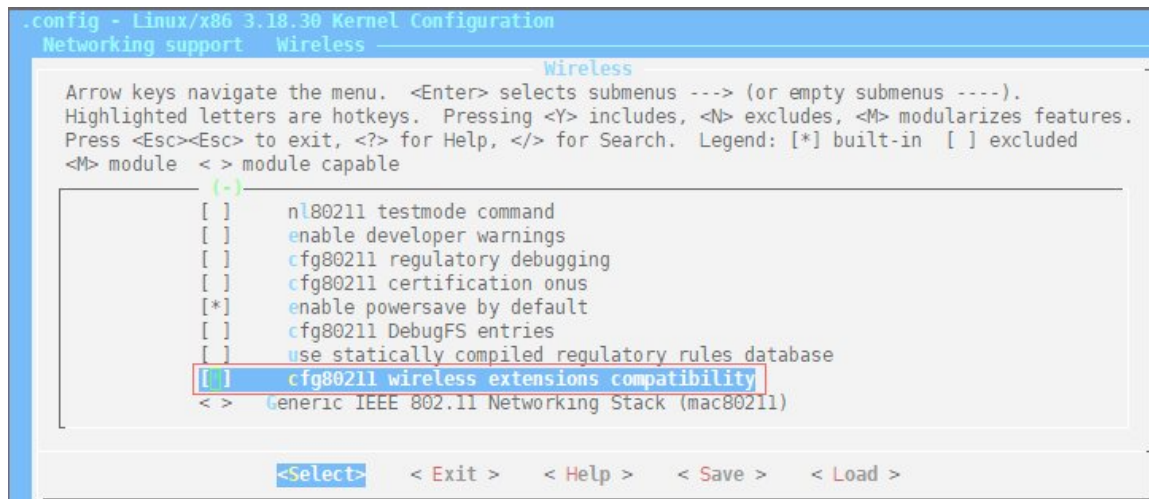


SSD_WLAN usage reference

1. Kernel configuration instructions

1.1. Configuring WEXT

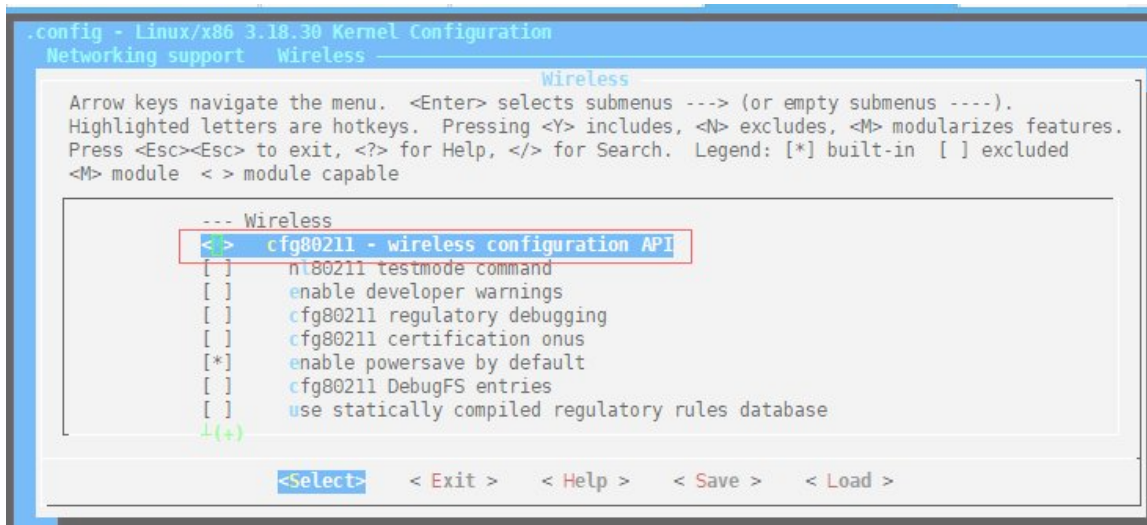
Wext is Wireless Extension, which is the standard interface of WIFI driver and user mode process in the kernel. The iwpriv command depends on this configuration.



```
.config - Linux/x86 3.18.30 Kernel Configuration
Networking support  Wireless  Wireless
Arrow keys navigate the menu. <Enter> selects submenus ---> (or empty submenus ----).
Highlighted letters are hotkeys. Pressing <Y> includes, <N> excludes, <M> modularizes features.
Press <Esc><Esc> to exit, <?> for Help, </> for Search. Legend: [*] built-in [ ] excluded
<M> module < > module capable
[ ] nl80211 testmode command
[ ] enable developer warnings
[ ] cfg80211 regulatory debugging
[ ] cfg80211 certification onus
[*] enable powersave by default
[ ] cfg80211 DebugFS entries
[ ] use statically compiled regulatory rules database
[ ] cfg80211 wireless extensions compatibility
< > Generic IEEE 802.11 Networking Stack (mac80211)
<Select> < Exit > < Help > < Save > < Load >
```

1.2. Configuring CFG80211

CFG80211 is the standard interface of WIFI driver and user mode process in the kernel. Before CFG80211 appeared, it was WEXT, and now more and more CFG80211 is used.



1.3. SDIO WIFI (SSW102B) configuration

SDIO wifi hot-plug is not supported.

infinity2m.dtsi dts configuration:

```
sdmmc {
    compatible = "sstar,sdmmc";

    slotnum = <1>; // Takoyaki 只有一个sd卡槽，填1即可
    revcdz = <0>;

    slot-ip-orders = <0>, <1>, <2>;
    slot-pad-orders = <0>, <1>, <2>;
    slot-max-clks = <48000000>, <48000000>, <48000000>;
    slot-intcdzs = <0>, <1>, <1>; // Takoyaki 只有一个卡槽，改第一个即可
    slot-fakecdzs = <1>, <0>, <0>;
    slot-cdzs-gpios = <PAD_PM_SD_CDZ>, <0>, <0>;
    slot-pwr-gpios = <PAD_TTL0>, <0>, <0>; // ssc010a: PAD_TTL0 P2_ssc011a: PAD_GPI00
    slot-pwr-off-delay = <30>, <30>, <30>;
    slot-sdio-use = <1>, <0>, <0>; // 使能SDIO

    interrupts-extended = <ms_main_intc GIC_SPI INT_IRQ_SDIO IRQ_TYPE_LEVEL_HIGH>, //was INT_IRQ_SDIO
        <ms_main_intc GIC_SPI INT_IRQ_SDIO IRQ_TYPE_LEVEL_HIGH>,
        <ms_main_intc GIC_SPI INT_FIQ_SD_CDZ IRQ_TYPE_LEVEL_HIGH>,
        <ms_gpi_intc 42>,
        <ms_main_intc GIC_SPI INT_FIQ_SD_CDZ IRQ_TYPE_LEVEL_HIGH>;
    interrupt-names = "mie0_irq", "mie1_irq", "cdz_slot0_irq", "cdz_slot1_irq", "cdz_slot2_irq";

    //clocks = <CLK_sdio>, <GATE_MCM_sdio>, <GATE_SRAM_sdio>;
    clocks = <CLK_sdio>, <CLK_sdio>, <CLK_sdio>,
        <CLK_sdio>, <CLK_sdio>, <CLK_sdio>,
        <CLK_sdio>, <CLK_sdio>, <CLK_sdio>;
    status = "ok";
};
```

If it is jumper verification, if the signal is poor, you can change clk to 12M to verify, the changes are as follows:

```
sdmmc {
    compatible = "sstar,sdmmc";

    slotnum = <1>;
    revcdz = <0>;

    slot-ip-orders = <0>, <1>, <2>;
    slot-pad-orders = <0>, <1>, <2>;
    slot-max-clks = <12000000>, <48000000>, <48000000>;
    slot-intcdzs = <0>, <1>, <1>;
    slot-fakecdzs = <1>, <0>, <0>;
    slot-cdzs-gpios = <PAD_PM_SD_CDZ>, <0>, <0>;
    slot-pwr-gpios = <PAD_TTL0>, <0>, <0>; // ssc010a: PAD_TTL0 P2_ssc011a: PAD_GPIO0
    slot-pwr-off-delay = <30>, <30>, <30>;
    slot-sdio-use = <1>, <0>, <0>;

    interrupts-extended = <ms_main_intc GIC_SPI INT_IRQ_SDIO IRQ_TYPE_LEVEL_HIGH>, //was INT_IRQ
    <ms_main_intc GIC_SPI INT_IRQ_SDIO IRQ_TYPE_LEVEL_HIGH>,
    <ms_main_intc GIC_SPI INT_FIQ_SD_CDZ IRQ_TYPE_LEVEL_HIGH>,
    <ms_gpi_intc 42>,
    <ms_main_intc GIC_SPI INT_FIQ_SD_CDZ IRQ_TYPE_LEVEL_HIGH>;
    interrupt-names = "mie0_irq", "mie1_irq", "cdz_slot0_irq", "cdz_slot1_irq", "cdz_slot2_irq";

    //clocks = <CLK_sdio>, <GATE_MCM_sdio>, <GATE_SRAM_sdio>;
    clocks = <CLK_sdio>, <CLK_sdio>, <CLK_sdio>,
    <CLK_sdio>, <CLK_sdio>, <CLK_sdio>,
    <CLK_sdio>, <CLK_sdio>, <CLK_sdio>;
    status = "ok";
};
```

An extra note:

If you use the wifi reset pin, you need to check the configuration of the reset pin.
If there is no additional gpio control reset pin, the following can be ignored.

在demo板上使用gpio0来控制sd的power enable脚位。有可能在新的设计图纸上，将GPIO0用来控制reset。

需要在infinity2m-ssc011a-s01a-padmux-display.dtsi中屏蔽

```
//<PAD_GPIO0 PINMUX_FOR_GPIO_MODE MDRV_PUSE_SDIO0_PWR > ,
```

然后再uboot上添加：gpio out 0 0 和 gpio out 0 1 来控制复位。

1. kernle里控制sd power 的gpio，其实是根据padmux里的MDRV_PUSE_SDIO0_PWR来找到对应哪个gpio口的，dts里的slot-pwr-gpios在padmux存在情况下，是没有作用的。

```

kernel > arch > arm > boot > dts > infinity2m-ssc011a-s01a-padmux-display.dtsi
64         <PAD_TTL19         PINMUX_FOR_TTL_MODE_1      MDRV_PUSE_TTL_DOUT19 >,
65         <PAD_TTL20         PINMUX_FOR_TTL_MODE_1      MDRV_PUSE_TTL_DOUT20 >,
66         <PAD_TTL21         PINMUX_FOR_TTL_MODE_1      MDRV_PUSE_TTL_DOUT21 >,
67         <PAD_TTL22         PINMUX_FOR_TTL_MODE_1      MDRV_PUSE_TTL_DOUT22 >,
68         <PAD_TTL23         PINMUX_FOR_TTL_MODE_1      MDRV_PUSE_TTL_DOUT23 >,
69         <PAD_TTL24         PINMUX_FOR_TTL_MODE_1      MDRV_PUSE_TTL_CLK >,
70         <PAD_TTL25         PINMUX_FOR_TTL_MODE_1      MDRV_PUSE_TTL_HSYNC >,
71         <PAD_TTL26         PINMUX_FOR_TTL_MODE_1      MDRV_PUSE_TTL_VSYNC >,
72         <PAD_TTL27         PINMUX_FOR_TTL_MODE_1      MDRV_PUSE_TTL_DE >,
73         <PAD_UART0_RX       PINMUX_FOR_UART0_MODE_1    MDRV_PUSE_UART0_RX >,
74         <PAD_UART0_TX       PINMUX_FOR_UART0_MODE_1    MDRV_PUSE_UART0_TX >,
75         //<PAD_UART1_RX >,
76         //<PAD_UART1_TX >,
77         //<PAD_SD_CLK       PINMUX_FOR_I2S_MODE_3      MDRV_PUSE_I2S_BCK >,
78         <PAD_GPIO0          PINMUX_FOR_GPIO_MODE       MDRV_PUSE_SDIO0_PWR >,
79         <PAD_SD_CLK          PINMUX_FOR_SDIO_MODE_1     MDRV_PUSE_SDIO0_CLK >,
80         //<PAD_SD_CMD       PINMUX_FOR_I2S_MODE_3      MDRV_PUSE_I2S_SDI >,
81         <PAD_SD_CMD          PINMUX_FOR_SDIO_MODE_1     MDRV_PUSE_SDIO0_CMD >,
82         //<PAD_SD_D0        PINMUX_FOR_I2S_MODE_3      MDRV_PUSE_I2S_WCK >,
83         <PAD_SD_D0           PINMUX_FOR_SDIO_MODE_1     MDRV_PUSE_SDIO0_D0 >,
84         <PAD_SD_D1           PINMUX_FOR_SDIO_MODE_1     MDRV_PUSE_SDIO0_D1>,
85         <PAD_SD_D2           PINMUX_FOR_SDIO_MODE_1     MDRV_PUSE_SDIO0_D2>,
86         //<PAD_SD_D3        PINMUX_FOR_I2S_MODE_3      MDRV_PUSE_I2S_SDO >,
87         <PAD_SD_D3           PINMUX_FOR_SDIO_MODE_1     MDRV_PUSE_SDIO0_D3>,
88         <PAD_PM_SD_CDZ       PINMUX_FOR_GPIO_MODE       MDRV_PUSE_SDIO0_CDZ>,
89         <PAD_PM_IRIN         PINMUX_FOR_PM_IRIN_MODE    MDRV_PUSE_IR>, // IR: default non-GPIO
90         //<PAD_PM_SPI_CZ >, // default not-GPIO
91         //<PAD_PM_SPI_CK >, // default not-GPIO
92         //<PAD_PM_SPI_DI >, // default not-GPIO
93         //<PAD_PM_SPI_DO >, // default not-GPIO

```

2. power不是硬件长供电的情况下，作为SDIO使用时，如果发生初始化的时候发生错误，注意要修改如下时间为30

```

kernel > arch > arm > boot > dts > infinity2m.dtsi
398         cmd-driving = <0>;
399         data-driving = <0>;
400         status = "ok";
401     };
402
403     sdmmc {
404         compatible = "sstar,sdmmc";
405
406         slotnum = <1>;
407         revcdz = <0>;
408
409         slot-ip-orders = <0>,<1>,<2>;
410         slot-pad-orders = <0>,<1>,<2>;
411         slot-max-clks = <48000000>,<48000000>,<48000000>;
412         slot-intcdzs = <0>,<1>,<1>;
413         slot-fakecdzs = <1>,<0>,<0>;
414         slot-cdzs-gpios = <PAD_PM_SD_CDZ>,<0>,<0>;
415         slot-pwr-gpios = <PAD_TTL0>,<0>,<0>; // ssc010a:PAD_TTL0 P2_ssc011a: PAD_GPIO0
416         slot-pwr-off-delay = <30>,<30>,<30>;
417         slot-pwr-on-delay = <30>,<1>,<1>;
418         slot-sdio-use = <1>,<0>,<0>;
419
420         interrupts-extended = <&ms_main_intc GIC_SPI INT_IRQ_SDIO IRQ_TYPE_LEVEL_HIGH>,//was INT_IRQ_SDIO
421         | <&ms_main_intc GIC_SPI INT_IRQ_SDIO IRQ_TYPE_LEVEL_HIGH>,
422         | <&ms_main_intc GIC_SPI INT_FIQ_SD_CDZ IRQ_TYPE_LEVEL_HIGH>,
423         | <&ms_gpi_intc 42>,
424         | <&ms_main_intc GIC_SPI INT_FIQ_SD_CDZ IRQ_TYPE_LEVEL_HIGH>;
425         interrupt-names = "mie0_irq", "mie1_irq", "cdz_slot0_irq", "cdz_slot1_irq", "cdz_slot2_irq";
426
427         //clocks = <&CLK_edion &GATE_MCM_edion &GATE_SRAM_edion>

```

2. 工具使用

2.1. wap_supplicant使用

wpa_supplicant是一个独立运行的守护进程，用来启动无线网络后台服务，在消息循环中处理WPA状态机、控制命令、驱动事件、配置信息等。

常用命令参数如下：

```
-I <ifname>          // 网络接口名称
-c <conf>            // 配置文件名称
-C <ctrl_intf>       // 控制接口名称
-D<driver>          // 驱动类型名称
-p <driver_param>    // 驱动参数
-b <br_ifname>       // 桥接口名称
-d                  // 增加调试信息
```

使用样例：

```
/config/wifi/wpa_supplicant -Dnl80211 -i wlan0 -c
/customer/wpa_supplicant.conf -d &
```

启动无线网络服务，驱动类型为nl80211，使用网络端口名称为wlan0，配置文件路径为/customer/wpa_supplicant.conf。若配置文件中保存有热点信息，则发起连接。

2.2. wpa_cli使用

wpa_cli是客户端程序，与wpa_supplicant通信，搜索、设置和连接网络。

常用命令参数如下（如当前网络接口为wlan0）：

```
wpa_cli -i wlan0 scan          // 扫描附近的热点
wpa_cli -i wlan0 scan_result   // 显示扫描结果
wpa_cli -I wlan0 status        // 查看当前连接信息
wpa_cli -I wlan0 add_network    // 新建一个链接，返回network id
```

```
wpa_cli -I wlan0 set_network <network id> <variable> <value> // 设置网络参数，如ssid，psk，keymgmt

wpa_cli -I wlan0 select_network <network id> // 选择指定的网络（会断开其它连接）

wpa_cli -I wlan0 enable_network <network id> // 使能指定网络

wpa_cli -I wlan0 disable_network <network id> // 禁用指定网络

wpa_cli -I wlan0 save_config // 保持连接信息至
wpa_supplicant.conf
```

2.3. iwlist使用

iwlist用于对/proc/net/wireless文件进行分析，得出无线网卡相关信息。

常用命令参数如下（如当前网络接口为wlan0）：

```
iwlist wlan0 scanning // 扫描当前无线网络

iwlist wlan0 frequen // 显示频道信息

iwlist wlan0 rate // 显示连接速度

iwlist wlan0 power // 显示电源模式
```

2.4. udhcpc使用

udhcpc作为dhcp客户端，动态获取ip。

常用命令参数如下：

```
-i <ifname> // 网络接口名称

-s <script> // udhcpc script

-a // 使用arping验证提供的地址

-t <count> // 发送数据包次数
```

```
-T <time>          // 数据包间时间间隔 ( 默认是3s )

-f                // 前台执行

-b                // 后台执行

-n                // 租约获取失败时退出

-q                // 获取到租约后退出
```

使用样例：

```
udhcpc -q -i wlan0 -s /etc/init.d/udhcpc.script &
```

2.5. hostapd使用

hostapd用来将当前设备作为热点，允许其它网络设备接入。

使用样例：

```
hostapd -B /config/wifi/hostapd.conf    // -B：在后台执行守护进程
```

2.6. dnsmasq使用

dnsmasq是一个轻量级的，易于配置的DNS转发器和DHCP服务器。它的目的是给一个小网络提供DNS和可选的DHCP。

常用命令参数如下：

```
-i <ifname> // 网络接口名称

-C <path>   // 指定配置文件路径 ( 默认为/etc/dnsmasq.conf )

--no-daemon // 非后台执行
```

使用样例：

```
dnsmasq -i p2p0 --no-daemon -C /config/wifi/dnsmasq.conf &
```

2.7. hostapd_cli使用

hostapd_cli作为hostapd的客户端，使用时，首先需要先启动hostapd主程序。
hostapd启动后，运行cli客户端时，会自动去连接当前正在工作的hostapd进程，
连接成功后，cli客户端就可以对hostapd应用程序进行参数的获取和控制。

常用命令参数如下：

```
-i <ifname> // 网络接口名称  
-p <path>    // 控制套接字路径（默认为/var/run/hostapd）  
-B           // 后台执行守护进程
```

使用样例：

```
/config/wifi/hostapd_cli -I p2p0 -p /var/run/hostapd all_sta &
```

3. WIFI功能测试

3.1. 加载WIFI驱动

先insmod加载WIFI驱动，以ssw101b为例，对应驱动是
ssw101b_wifi_HT40_usb.ko。执行insmod ssw101b_wifi_HT40_usb.ko，使用
ifconfig -a查看，可以看到新生成wlan0节点和p2p0节点：


```

/config/wifi # ifconfig -a
eth0      Link encap:Ethernet  HWaddr 00:70:27:00:00:01
          inet addr:172.19.24.186  Bcast:172.19.24.255  Mask:255.255.255.0
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:74 errors:0 dropped:0 overruns:0 frame:0
          TX packets:27 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:11028 (10.7 KiB)  TX bytes:1134 (1.1 KiB)
          Interrupt:35

eth1      Link encap:Ethernet  HWaddr 00:30:1B:BA:02:DB
          BROADCAST MULTICAST  MTU:1500  Metric:1
          RX packets:0 errors:0 dropped:0 overruns:0 frame:0
          TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:0 (0.0 B)  TX bytes:0 (0.0 B)
          Interrupt:37

lo        Link encap:Local Loopback
          inet addr:127.0.0.1  Mask:255.0.0.0
          UP LOOPBACK RUNNING  MTU:65536  Metric:1
          RX packets:9 errors:0 dropped:0 overruns:0 frame:0
          TX packets:9 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1
          RX bytes:864 (864.0 B)  TX bytes:864 (864.0 B)

p2p0     Link encap:Ethernet  HWaddr 26:14:07:0B:D8:14
          BROADCAST MULTICAST  MTU:1500  Metric:1
          RX packets:0 errors:0 dropped:0 overruns:0 frame:0
          TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:0 (0.0 B)  TX bytes:0 (0.0 B)

wlan0    Link encap:Ethernet  HWaddr 24:14:07:0B:D8:14
          BROADCAST MULTICAST  MTU:1500  Metric:1
          RX packets:0 errors:0 dropped:0 overruns:0 frame:0
          TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:0 (0.0 B)  TX bytes:0 (0.0 B)

/config/wifi #

```

3.2. STA模式测试

```

export
LD_LIBRARY_PATH=$LD_LIBRARY_PATH:/usr/local/lib:/lib:/config/wifi
mkdir -p /tmp/wifi/run
chmod 777 /tmp/wifi/run
ifconfig wlan0 up

```

创建wpa_supplicant.conf文件，编辑如下，使用测试热点为“WiFi_test”，密码为“12345678”。

```
/config/wifi # cat /appconfigs/wpa_supplicant.conf
ctrl_interface=/tmp/wifi/run/wpa_supplicant
update_config=1

network={
    ssid="WiFi_test"
    psk="12345678"
}
```

测试连接：

```
/config/wifi/wpa_supplicant -Dnl80211 -i wlan0 -c
/appconfigs/wpa_supplicant.conf -d &
```

获取IP：

```
udhcpc -q -i wlan0 -s /etc/init.d/udhcpc.script &
```

查看wlan0信息：

```
/config/wifi # ifconfig wlan0
wlan0      Link encap:Ethernet  HWaddr 24:14:07:0B:D8:14
            inet addr:192.168.1.104  Bcast:192.168.1.255  Mask:255.255.255.0
            UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
            RX packets:32 errors:0 dropped:2 overruns:0 frame:0
            TX packets:4 errors:0 dropped:0 overruns:0 carrier:0
            collisions:0 txqueuelen:1000
            RX bytes:6286 (6.1 KiB)  TX bytes:980 (980.0 B)
```

扫描热点：

```
/config/wifi/iwlist wlan0 scan | grep Cell -A 5
```

```
RTM_NEWLINK; operstate=1 ifi_flags=0x11043 ([UP][RUNNING][LOWER_UP])
RTM_NEWLINK; IFLA_IFNAME: Interface 'wlan0' added
nl80211: if removed already cleared - ignore event
```

```
Cell 01 - Address: 64:6E:97:C3:13:F0
Channel:1
Frequency:2.412 GHz (Channel 1)
Quality=46/70 Signal level=-64 dBm
Encryption key:on
ESSID:"WiFi_test"
```

```
--
Cell 02 - Address: 3C:06:A7:B1:E6:5B
Channel:6
Frequency:2.437 GHz (Channel 6)
Quality=24/70 Signal level=-86 dBm
Encryption key:off
ESSID:"TP-LINK_E65B"
```

```
--
Cell 03 - Address: A0:63:91:E4:5A:33
Channel:11
Frequency:2.462 GHz (Channel 11)
Quality=36/70 Signal level=-74 dBm
Encryption key:on
ESSID:"SSTAR_Guest"
```

```
--
Cell 04 - Address: 00:6B:8E:F0:36:90
Channel:13
Frequency:2.472 GHz (Channel 13)
Quality=29/70 Signal level=-81 dBm
Encryption key:on
ESSID:"PDCN"
```

```
--
Cell 05 - Address: 50:FA:84:27:FF:A2
Channel:12
Frequency:2.467 GHz (Channel 12)
Quality=20/70 Signal level=-90 dBm
Encryption key:on
ESSID:"guanhong_ap"
```

```
--
Cell 06 - Address: 3A:6B:5B:D1:83:15
Channel:11
Frequency:2.462 GHz (Channel 11)
Quality=19/70 Signal level=-91 dBm
Encryption key:on
ESSID:"Sigmaster_2.4G"
```

可以看到，当前连接的热点“WiFi_test”信道为1。

3.3. AP模式测试

```
mkdir -p /var/run/hostapd
```

```
mkdir -p /var/wifi/misc
```

```
mkdir -p /var/lib/misc
```

```
ifconfig p2p0 up
```

```
ifconfig p2p0 192.168.1.100 netmask 255.255.255.0
```

hostapd配置文件在/config/wifi/hostapd.conf，其中ssid为当前设备的热点名称，wpa_passphrase为设备的连接密码。可修改此文件变更设备名称和密码。

开启热点：

```
/config/wifi/hostapd -B /config/wifi/hostapd.conf
```

```
/config/wifi/dnsmasq -i p2p0 --no-daemon -C  
/config/wifi/dnsmasq.conf &
```

网络设备可以搜索到热点“ssw101bap”，使用手机连接该热点：



查看设备连接信息：

```
/config/wifi/hostapd_cli -i p2p0 -p /var/run/hostapd all_sta &
```

```
/config/wifi # /config/wifi/hostapd_cli -i p2p0 -p /var/run/hostapd all_sta &
/config/wifi # 12:10:19:6f:c3:9c
flags=[AUTH][ASSOC][AUTHORIZED][SHORT_PREAMBLE]
aid=1
capability=0x431
listen_interval=20
supported_rates=82 84 8b 96 24 30 48 6c 0c 12 18 60
timeout_next=NULLFUNC POLL
dot11RSNAStatsSTAAddress=12:10:19:6f:c3:9c
dot11RSNAStatsVersion=1
dot11RSNAStatsSelectedPairwiseCipher=00-0f-ac-4
dot11RSNAStatsTKIPLocalMICFailures=0
dot11RSNAStatsTKIPRemoteMICFailures=0
hostapdWPAPTKState=11
hostapdWPAPTKGroupState=0
rx_packets=906
tx_packets=8
rx_bytes=40474
tx_bytes=1018
connected_time=62
```

可以看到接入数量为 1，接入设备mac与手机mac一致。

3.4. 桥接模式测试

WIFI同时开启STA和AP的情况下，在wlan0和p2p0之间建立桥接。

Kernel配置添加网络桥接支持，编译kernel在kernel/modules目录会生成llc.ko，stp.ko，bridge.ko。网络桥接前依次加载这三个驱动。注：Kernel网络相关配置变化，wifi驱动可能需要重新编译。

```
Networking support  Networking options
Networking options
Arrow keys navigate the menu. <Enter> selects submenus ---> (or empty submenus ---
Pressing <Y> includes, <N> excludes, <M> modularizes features. Press <Esc><Esc> to
Legend: [*] built-in [ ] excluded <M> module < > module capable

-* Packet socket
< > Packet: sockets monitoring interface
<*> Unix domain sockets
< > UNIX: socket monitoring interface
< > Transformation user configuration interface
< > PF_KEY sockets
-* TCP/IP networking
[*] IP: multicasting
[*] IP: advanced router
[ ] FIB TRIE statistics
[ ] IP: policy routing
[*] IP: equal cost multipath
[*] IP: verbose route monitoring
[ ] IP: kernel level autoconfiguration
< > IP: tunneling
< > IP: GRE demultiplexer
[ ] IP: multicast routing
[*] IP: TCP syncookie support
< > IP: Foo (IP protocols) over UDP
< > IP: AH transformation
< > IP: ESP transformation
< > IP: IPComp transformation
< > IP: IPsec transport mode
< > IP: IPsec tunnel mode
< > IP: IPsec BEET mode
< > INET: socket monitoring interface
[*] TCP: advanced congestion control --->
[ ] TCP: MD5 Signature Option support (RFC2385)
< > The IPv6 protocol ----
[ ] Security Marking
[ ] Timestamping in PHY devices
[ ] Network packet filtering framework (Netfilter) ----
< > The DCCP Protocol ----
< > The SCTP Protocol ----
< > The RDS Protocol
< > The TIPC Protocol ----
< > Asynchronous Transfer Mode (ATM)
< > Layer Two Tunneling Protocol (L2TP) ----
<M> 802.1d Ethernet Bridging
[*] IGMP/MLD snooping
< > Distributed Switch Architecture
< > 802.1Q/802.1ad VLAN Support
< > DECnet Support
```

建立桥接还需要需编辑/config/wifi/hostapd.conf，添加bridge字段，并修改channel，使之与STA模式接入热点的信道相同。

```

/ # cat /config/wifi/hostapd.conf
interface=p2p0
bridge=br0
ctrl_interface=/var/run/hostapd
ctrl_interface_group=0
driver=nl80211
ssid=ssw101bap
hw mode=g
channel=1
macaddr_acl=0
auth_algs=1
ignore_broadcast_ssid=0
wpa=3
wpa_passphrase=12345678
wpa_key_mgmt=WPA-PSK
wpa_pairwise=TKIP
rsn_pairwise=CCMP/ #
/ #

```

编译kernel和wifi驱动后，按3.1节加载wifi驱动，再按3.2节和3.3节的步骤启用STA模式接入测试热点“WiFi_test”，并启用AP模式开启热点。

建立桥接：

```

brctl addbr br0

brctl addif br0 wlan0

brctl addif br0 p2p0

ifconfig br0 up

```

使用测试手机接入热点“ssw101bap”。在“WiFi_test”接入外网的情况下，测试手机可正常访问外网。

查看br0信息：

```

/config/wifi # ifconfig br0
br0      Link encap:Ethernet  HWaddr 24:14:07:0B:D8:14
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:526 errors:0 dropped:95 overruns:0 frame:0
          TX packets:18 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:106509 (104.0 KiB)  TX bytes:756 (756.0 B)

```

4. MI_WLAN使用示例

4.1. MI_WLAN在STA模式下连接热点

本地设备连接AP示例：

```
1.     static WLAN_HANDLE g_wlanHdl = -1;
2.     static MI_WLAN_InitParams_t g_stParam =
3.     {"/config/wifi/wlan.json"};
4.     static MI_WLAN_OpenParams_t g_stOpenParam =
5.     {E_MI_WLAN_NETWORKTYPE_INFRA};
6.     static MI_WLAN_ConnectParam_t g_stConnectParam = {
7.         E_MI_WLAN_SECURITY_WPA,
8.         "WiFi_test",
9.         "12345678",
10.        5000
11.    };
12. static bool g_bConnected = false;
13. static pthread_t g_ptConn = NULL;
14. static bool g_bThreadRun = false;
15. void *_ConnectWorkThread(void *args)
16. {
17.     int nTimeoutCnt = 50;           // timeout 10s
18.     MI_WLAN_Status_t status;
19.     MI_WLAN_ConnectParam_t *pstConnParam =
20.     (MI_WLAN_ConnectParam_t*)args;
21.     MI_WLAN_Connect(&g_wlanHdl, pstConnParam);
22.     printf("current conn info: handle=%d, ssid=%s,
23.     passwd=%s\n", g_wlanHdl, (char*)pstConnParam->au8SSID,
24.     (char*)pstConnParam->au8Password);
25.     while (1)
26.     {
27.         if (!g_bThreadRun)
28.             break;
29.         if (nTimeoutCnt-- <= 0)
30.         {
31.             printf("connect failed, timeout\n");
32.             break;
33.         }
34.         static int i = 0;
35.         MI_WLAN_GetStatus(g_wlanHdl, &status);
36.         printf("MI_WLAN_GetStatus: count %d\n", i++);
37.         if (status.stStaStatus.state == WPA_COMPLETED)
```



```

39.         {
40.             g_bConnected = true;
41.             printf("connect success: ssid=%s, ip=%s\n",
(char*)status.stStaStatus.ssid,
(char*)status.stStaStatus.ip_address);
42.             break;
43.         }
44.
45.         usleep(100000);
46.     }
47.
48.     return NULL;
49. }
50.
51. int main(int argc, char **argv)
52. {
53.     MI_WLAN_ScanResult_t scanResult;
54.     char ch = 0;
55.
56.     // init wlan module
57.     if (MI_WLAN_Init(&g_stParam))
58.     {
59.         printf("Wlan init failed, please ensure wlan module is
enabled.\n");
60.         return -1;
61.     }
62.
63.     if (MI_WLAN_Open(&g_stOpenParam))
64.     {
65.         printf("wlan open failed\n");
66.         break;
67.     }
68.
69.     g_bThreadRun = true;
70.     pthread_create(&g_ptConn, NULL, _ConnectWorkThread,
&g_stConnectParam);
71.
72.     while ((ch = getchar()) != 'q')
73.     {
74.         if (ch == 's')
75.         {
76.             memset(&scanResult, 0,
sizeof(MI_WLAN_ScanResult_t));
77.             MI_WLAN_Scan(NULL, &scanResult);
78.
79.             if (scanResult.u8APNumber > 0)
80.             {
81.                 printf("Scan result:\n");
82.

```

```

83.         for (int i = 0; i < scanResult.u8APNumber; i++)
84.         {
85.             char *pSsid =
(      (char*)scanResult.stAPIInfo[i].au8SSID;
86.             if (pSsid && strcmp(pSsid, "\"\""))
87.             {
88.                 char trimSsid[36];
89.                 memset(trimSsid, 0, sizeof(trimSsid));
90.                 strncpy(trimSsid, pSsid+1,
strlen(pSsid));
91.                 printf("SSID: %s\n", trimSsid);
92.                 printf("MAC: %s\n",
(char*)scanResult.stAPIInfo[i].au8Mac);
93.                 printf("encrypt: %s\n",
scanResult.stAPIInfo[i].bEncryptKey?"true":"false");
94.                 printf("signalSTR: %d db\n",
scanResult.stAPIInfo[i].stQuality.signalSTR);
95.                 printf("frequency: %f GHz\n",
scanResult.stAPIInfo[i].fFrequency);
96.                 printf("bitrate: %f Mb/s\n",
scanResult.stAPIInfo[i].fBitRate);
97.                 printf("channel: %d\n",
scanResult.stAPIInfo[i].u8Channel);
98.                 printf("channel: %d\n",
scanResult.stAPIInfo[i].u16CellId);
99.                 printf("\n");
100.            }
101.        }
102.    }
103. }
104. }
105.
106.     g_bThreadRun = false;
107.     pthread_join(g_ptConn, NULL);
108.
109.     if (g_bConnected)
110.     {
111.         MI_WLAN_Disconnect(g_wlanHdl);
112.         g_bConnected = false;
113.     }
114.
115.     MI_WLAN_Close(&g_stOpenParam);
116.     MI_WLAN_DeInit();
117.
118.     return 0;
119. }

```

4.2. MI_WLAN在AP模式下开启热点

本地设备作为host示例：

```
1. static WLAN_HANDLE g_apHdl = AP_HANDLE;
2. static MI_WLAN_OpenParams_t g_stOpenParam =
{E_MI_WLAN_NETWORKTYPE_AP};
3. static MI_WLAN_InitParams_t g_stParam =
{"/config/wifi/wlan.json"};
4. static MI_WLAN_Status_t g_stStatus;
5. static bool g_bConnected = false;
6.
7. int main(int argc, char **argv)
8. {
9.     // init wlan module
10.    if (MI_WLAN_Init(&g_stParam))
11.    {
12.        printf("Wlan init failed, please ensure wlan module is
enabled.\n");
13.        return -1;
14.    }
15.
16.    if (MI_WLAN_Open(&g_stOpenParam))
17.    {
18.        printf("wlan open failed\n");
19.        break;
20.    }
21.
22.    if (MI_SUCCESS == MI_WLAN_Connect(&g_apHdl, NULL))
23.        g_bConnected = true;
24.
25.    while ((ch = getchar()) != 'q')
26.    {
27.        if (ch == 's')
28.        {
29.            MI_WLAN_GetStatus(AP_HANDLE, &g_stStatus);
30.            if (g_stStatus.stApStatus.u16HostNum > 0)
31.            {
32.                printf("Access port:\n");
33.
34.                for (int i = 0; i <
g_stStatus.stApStatus.u16HostNum; i++)
35.                {
36.                    printf("HostName: %s\n",
g_stStatus.stApStatus.astHosts[i].hostname);
37.                    printf("IP: %s\n",
(char*)g_stStatus.stApStatus.astHosts[i].ipaddr);
```

```

38.             printf("MAC: %s\n",
(char*)g_stStatus.stApStatus.astHosts[i].macaddr);
39.             printf("Connected time: %lld\n",
g_stStatus.stApStatus.astHosts[i].connectedtime);
40.             printf("\n");
41.         }
42.     }
43. }
44. }
45.
46.     if (g_bConnected)
47.     {
48.         MI_WLAN_Disconnect(g_apHdl);
49.         g_bConnected = false;
50.     }
51.
52.     MI_WLAN_Close(&g_stOpenParam); 53. MI_WLAN_DeInit (); 54.
55. return 0 ; 56. 57. }

```

5. DEMO reference

MI_WLAN DemoCode reference:

[wifi.tar.gz](#) [media/wifi.tar.gz]

[wlan.json](#) [media/wlan.json]